

ON MULTIDIMENSIONAL PACKING PROBLEMS*

CHANDRA CHEKURI[†] AND SANJEEV KHANNA[‡]

Abstract. We study the approximability of multidimensional generalizations of three classical packing problems: multiprocessor scheduling, bin packing, and the knapsack problem. Specifically, we study the *vector scheduling* problem, its dual problem, namely, the *vector bin packing* problem, and a class of *packing integer programs*. The vector scheduling problem is to schedule n d -dimensional tasks on m machines such that the maximum load over all dimensions and all machines is minimized. The vector bin packing problem, on the other hand, seeks to minimize the number of bins needed to schedule all n tasks such that the maximum load on any dimension across all bins is bounded by a fixed quantity, say, 1. Such problems naturally arise when scheduling tasks that have multiple resource requirements. Finally, packing integer programs capture a core problem that directly relates to both vector scheduling and vector bin packing, namely, the problem of packing a maximum number of vectors in a single bin of unit height. We obtain a variety of new algorithmic as well as inapproximability results for these three problems.

Key words. multidimensional packing, vector scheduling, vector bin packing, packing integer programs, multiprocessor scheduling, bin packing, knapsack, approximation algorithms, hardness of approximation, combinatorial optimization

AMS subject classification. 68Q25

DOI. 10.1137/S0097539799356265

1. Introduction. Multiprocessor scheduling, bin packing, and the knapsack problem are very well studied problems in combinatorial optimization. Their study has had a large impact on the design and analysis of approximation algorithms. All of these problems involve packing items of different sizes into bins of finite capacities. In this work we study *multidimensional* generalizations of these problems where the items to be packed are d -dimensional vectors and bins are d -dimensional objects as well. We obtain a variety of approximability and inapproximability results in the process, significantly improving upon earlier known results for these problems. Some of our results include a polynomial time approximation scheme (PTAS) for the vector scheduling problem when the dimension is fixed, and an approximation algorithm for the vector bin-packing problem that improves a two-decade-old bound. Though our primary motivation is vector scheduling and vector bin packing, an underlying problem that arises is the problem of maximizing the numbers of vectors that can be packed into a bin of fixed size. This is a special case of the multidimensional knapsack problem that is equivalent to packing integer programs (PIPs) [27, 29]. PIPs are an important class of integer programs that capture several NP-hard combinatorial optimization problems including the maximum independent set problem, the disjoint

*Received by the editors May 17, 1999; accepted for publication (in revised form) December 22, 2003; published electronically May 5, 2004. A preliminary version of this paper appeared in the *Proceedings of the 10th Annual ACM-SIAM Symposium on Discrete Algorithms*, Baltimore, MD, 1999, pp. 185–194.

<http://www.siam.org/journals/sicomp/33-4/35626.html>

[†]Bell Labs, 600-700 Mountain Ave., Murray Hill, NJ 07974 (chekuri@research.bell-labs.com). Most of this work was done during a summer internship at Bell Labs while the author was at Stanford University. Work at Stanford was supported by an IBM Cooperative fellowship, an ARO MURI grant DAAH04-96-1-0007, and NSF Award CCR-9357849.

[‡]Department of CIS, University of Pennsylvania, Philadelphia, PA 19104 (sanjeev@cis.upenn.edu). Most of this research was done when the author was at Bell Labs, Lucent Technologies. The research of this author was supported in part by an Alfred P. Sloan Research Fellowship and an NSF Career Award CCR-0093117.

paths problem, and hypergraph matchings. The only general technique known for approximating PIPs is to use randomized rounding on the natural LP relaxation [27, 29]. We show here that the approximation guarantee for PIPs, as obtained via randomized rounding, is essentially the best possible unless $\text{NP} = \text{ZPP}$.

In addition to their theoretical importance, these problems have several applications such as load balancing, cutting stock, and resource allocation, to name a few. One of our motivations for studying these problems comes from recent interest [12, 13, 14] in multidimensional resource scheduling problems in parallel query optimization. A favored architecture for parallel databases is the so-called shared-nothing environment [4] where the parallel system consists of a set of independent processing units each of which has a set of time-sharable resources such as CPU, one or more disks, network controllers, etc. A task executing on one of these units has requirements from each of these resources and is best described as a multidimensional load vector. However in most work on scheduling, both in theory and practice, it is assumed that the load of a task is described by a single aggregate work measure. This simplification is done typically to reduce the complexity of the scheduling problem. However, for large task systems that are typically encountered in database applications, ignoring the multidimensionality could lead to bad performance. The work in [11, 12, 13, 14] demonstrates the practical effectiveness of the multidimensional approach. One of the basic resource scheduling problems that is considered in the above papers is the problem of scheduling d -dimensional vectors (tasks) on d -dimensional bins (machines) to minimize the maximum load on any dimension (the load on the most loaded resource). Surprisingly, despite the large body of work on approximation algorithms for multiprocessor scheduling and its several variants [15, 23], the authors in [11] had to settle for a naive $(d + 1)$ -approximation for the d -dimensional vector scheduling problem. Our work here provides a PTAS when d is fixed and an $O(\ln^2 d)$ -approximation when d is arbitrary. A similar situation existed for the vector bin packing problem where the best known approximation ratio prior to our work was $(d + \epsilon)$. In this paper, we improve this to obtain a $(1 + \epsilon \cdot d + O(\ln \epsilon^{-1}))$ -approximation for any fixed $\epsilon > 0$. In what follows, we formally define the problems that we study and provide a detailed description of our results.

1.1. Problem definitions. We start by defining the vector scheduling problem. For a vector x , the quantity $\|x\|_\infty$ denotes the standard ℓ_∞ norm.

DEFINITION 1.1 (vector scheduling (VS)). *We are given a set J of n rational d -dimensional vectors $p_1, \dots, p_n$ from $[0, \infty)^d$ and a number m . A valid solution is a partition of J into m sets $A_1, \dots, A_m$. The objective is to minimize $\max_{1 \leq i \leq m} \|\bar{A}_i\|_\infty$, where $\bar{A}_i = \sum_{j \in A_i} p_j$ is the sum of the vectors in A_i .*

DEFINITION 1.2 (vector bin packing (VBP)). *Given a set of n rational vectors $p_1, \dots, p_n$ from $[0, 1]^d$, find a partition of the set into sets $A_1, \dots, A_m$ such that $\|\bar{A}_i\|_\infty \leq 1$ for $1 \leq i \leq m$. The objective is to minimize m , the size of the partition.*

The following definition of PIPs is from [29]. In the literature this problem is also referred to as the d -dimensional 0-1 knapsack problem [7].

DEFINITION 1.3 (packing integer program (PIP)). *Given $A \in [0, 1]^{d \times n}$, $b \in [1, \infty)^d$, and $c \in [0, 1]^n$ with $\max_j c_j = 1$, a PIP seeks to maximize $c^T x$ subject to $x \in \{0, 1\}^n$ and $Ax \leq b$. Furthermore if $A \in \{0, 1\}^{d \times n}$, b is assumed to be integral. Finally, B is defined to be $\min_i b_i$.*

The restrictions on A , b , and c in the above definition are without loss of generality: an arbitrary packing problem can be reduced to the above form (see [29]). We are interested in PIPs where $b_i = B$ for $1 \leq i \leq d$. When $A \in \{0, 1\}^{d \times n}$ this

problem is known as the *simple B -matching in hypergraphs* [24]: given a hypergraph with nonnegative edge weights, find a maximum weight collection of edges such that no vertex occurs in more than B of them. When $B = 1$ this is the usual hypergraph matching problem. We note that the maximum *independent set* problem in graphs is a special case of the hypergraph matching problem.

1.2. Related work and our results. All the problems we consider are NP-complete for $d = 1$ (multiprocessor scheduling, bin packing, and the knapsack problem). The dimension of the vectors, d , plays an important role in determining the complexity. We concentrate on two cases, when d is a fixed constant, and when d is part of the input and can be arbitrary. Below is an outline of the various positive and negative results that we obtain for these problems.

Vector scheduling. For the VS problem the best approximation algorithm [13] prior to our work had a ratio of $(d+1)$. When d is a fixed constant (a case of practical interest) we obtain a PTAS, generalizing the result of Hochbaum and Shmoys [19] for multiprocessor scheduling. In addition we obtain a simpler $O(\ln d)$ -approximation algorithm that is better than $(d+1)$ for all $d \geq 2$. When d is large we give an $O(\ln^2 d)$ -approximation that uses known approximation algorithms for PIPs as a subroutine. We also give a very simple $O(\ln dm / \ln \ln dm)$ -approximation. Finally, we show that it is hard to approximate the VS problem to within *any* constant factor when d is arbitrary.

Vector bin packing. The previous best known approximation algorithms for this problem gave a ratio of $(d + \epsilon)$ for any fixed $\epsilon > 0$ [6] and $(d + 7/10)$ [9]; the latter result holds even in an online setting. All the ratios mentioned are asymptotic; that is, there is an additive term depending on d and on ϵ . Karp, Luby, and Marchetti-Spaccamela [22] do a probabilistic analysis and show bounds on the average wastage in the bins. We design an approximation algorithm that for any fixed $\epsilon > 0$, achieves a $(1 + \epsilon \cdot d + O(\ln \epsilon^{-1}))$ -approximation in polynomial time, thus improving upon the previous guarantees. One useful corollary of this result is that when d is a fixed constant we can approximate the problem to within a ratio of $O(\ln d)$. When d is arbitrary a simple reduction from the graph coloring problem gives a $d^{\frac{1}{2}-\epsilon}$ hardness for any fixed $\epsilon > 0$ even when vectors are drawn from the set $[0, 1]^d$. Moreover, even when $d = 2$ the problem is APX-hard [31]; this is an interesting departure from the classical bin packing problem ($d = 1$) which exhibits an asymptotic FPTAS.

Packing integer programs. For fixed d there is a PTAS for PIPs [7]. For large d the randomized rounding technique of Raghavan and Thompson [27] yields integral solutions of value $t_1 = \Omega(\text{OPT}/d^{1/B})$ if $A \in [0, 1]^{d \times n}$, and $t_2 = \Omega(\text{OPT}/d^{1/(B+1)})$ if $A \in \{0, 1\}^{d \times n}$. Srinivasan [29] improved these results to obtain solutions of value $\Omega(t_1^{B/(B-1)})$ and $\Omega(t_2^{(B+1)/B})$, respectively (see discussion at the end of section 4.1 concerning when these values are better). Thus the parameter B plays an important role in the approximation ratio achieved, with better ratios obtained as B gets larger (recall that entries in A are upper bounded by 1). It is natural to question if the dependence of the approximation ratio on B could be any better. We show that PIPs are hard to approximate to within a factor of $\Omega(d^{\frac{1}{B+1}-\epsilon})$ for every fixed B , thus establishing that randomized rounding essentially gives the best possible approximation guarantees. Hardness was known only for the case $B = 1$ via a reduction from the maximum independent set problem. We show how this can be amplified to work for larger values of B and then use Hastad's result [18] on the inapproximability of the maximum independent set problem. An interesting aspect of our reduction is that the hardness result holds even when the optimal is restricted to choosing a solution

that satisfies $Ax \leq 1^d$ while the approximation algorithm is required to satisfy only the relaxed constraint of $Ax \leq B^d$.

Table 1.1 summarizes our results. For conciseness, in the table below when we indicate that a problem is c -hard we mean that, unless $\text{NP} = \text{ZPP}$, no polynomial time algorithm can approximate it to within a factor of c .

TABLE 1.1
Approximation bounds and inapproximability results for each problem.

Problem	$d = 1$	Constant $d \geq 2$	Arbitrary d
Vector Scheduling	PTAS [19]	$(d + 1)$ (folklore) PTAS (this paper)	$(d + 1)$ (folklore) $O(\ln^2 d)$ (this paper)
	NP-hard	NP-hard	NP-hard c -hard $\forall c > 1$ (this paper)
Vector Bin Packing	AFPTAS ¹ [6, 21]	$(d + \epsilon)$ [6] $O(\ln d)$ (this paper)	$(d + \epsilon)$ [6] $1 + \epsilon d + O(\ln \frac{1}{\epsilon})$ (this paper)
	NP-hard	APX-hard [31]	APX-hard [31] $d^{\frac{1}{2}-\epsilon}$ -hard (this paper)
Packing Integer Programs	FPTAS [20]	PTAS [7]	$O(d^{\frac{1}{B+1}})$ [26, 27, 29]
	NP-hard	NP-hard	$d^{\frac{1}{2}-\epsilon}$ -hard for $B = 1$ $d^{\frac{1}{B+1}-\epsilon}$ -hard $\forall B \geq 1$ (this paper)

1.3. Organization. The rest of the paper is organized as follows. Sections 2 and 3 present our approximation algorithms for the vector scheduling problem and the vector bin packing problem, respectively. In section 4 we present our inapproximability results for the three problems.

2. Algorithms for vector scheduling. For any set of vectors (jobs) A , we define $\bar{A}$ to be the vector sum $\sum_{j \in A} p_j$. The quantity $\bar{A}^i$ denotes component i of the vector $\bar{A}$. Throughout this section, we assume without loss of generality that the vectors have been scaled such that the optimal schedule value is 1.

2.1. A PTAS for fixed d . Hochbaum and Shmoys [19] gave a PTAS for the multiprocessor scheduling problem (VS problem with $d = 1$) using dual approximation schemes. We now show that a nontrivial generalization of their ideas yields a PTAS for arbitrary but fixed d .

The basic idea used in [19] is a primal-dual approach whereby the scheduling problem is viewed as a bin packing problem. If an optimal solution can pack all jobs with load not exceeding some height h (assume $h = 1$ from here on), then the scheduling problem is to pack all the jobs into m bins (machines) of height 1. The authors then give an algorithm to solve this bin packing problem with bin height relaxed to $(1 + \epsilon)$, where $\epsilon > 0$ is a fixed constant. In order to do so, they classify a job as large or small depending on whether its size is greater than ϵ or not. Only a fixed number (at most $1/\epsilon$) of large jobs can be packed into any bin. The sizes of the large jobs are then rounded up to be one of $O(\ln 1/\epsilon)$ distinct values. Dynamic programming is used to pack the rounded up large jobs into the m bins such that no

¹AFPTAS denotes an *asymptotic* FPTAS. A problem has an AFPTAS if for any $\epsilon > 0$, there exists a positive integer N_ϵ such that there is a $(1 + \epsilon)$ -approximation algorithm that runs in time polynomial in the input size and $1/\epsilon$ for all instances of the problem with optimum value at least N_ϵ .

bin exceeds a height of $(1 + \epsilon)$. The small jobs are then greedily packed on top of the large jobs.

We take a similar approach to the problem. Our dual problem is VBP. The primary difficulty in generalizing the above ideas to the case of jobs or vectors of $d \geq 2$ dimensions is the lack of a total order on the “size” of the jobs. It is still possible to classify a vector as large or small depending on its ℓ_∞ norm. However, the scheme of [19], whereby the small jobs are greedily packed on top of the large jobs, does not apply. We need to take into account the interaction between the packing of large and small vectors. In addition, the packing of small vectors is nontrivial. In fact we use a linear programming relaxation and a careful rounding to pack the small jobs. We describe our ideas in detail below. Following the above discussion we will think of machines as d -dimensional bins and the schedule length as bin capacity (height). Given an $\epsilon > 0$ and a guess for the optimal value (that we assume is normalized to 1), we describe an ϵ -relaxed decision procedure A_ϵ that either returns a schedule of height $(1 + \epsilon)$ or proves that the guess is incorrect. We can use A_ϵ to do a binary search for the optimal value. Let δ be ϵ/d .

Preprocessing step. Our first idea is to reduce to zero all coordinates of the vectors that are too small relative to the largest coordinate. This allows us to bound the ratio of the largest coordinate to the smallest nonzero coordinate.

LEMMA 2.1. *Let I be an instance of the VS problem. Let I' be a modified instance where we replace each p_j in I with a vector q_j as follows. For each $1 \leq i \leq d$, $q_j^i = p_j^i$ if $p_j^i > \delta \|p_j\|_\infty$ and $q_j^i = 0$ otherwise. Then, replacing the vector q_j by the vector p_j in any valid solution to I' results in a valid solution to I of height at most a factor of $(1 + \epsilon)$ that of I' .*

Proof. Let A be a set of vectors in I , and let B be the corresponding set of vectors in I' . Then it follows from the transformation described above that

$$\begin{aligned} \bar{A}^i &\leq \bar{B}^i + \delta \sum_{j \in B} \|q_j\|_\infty \\ &\leq \bar{B}^i + \delta \sum_{j \in B} \|q_j\|_1 \\ &\leq \bar{B}^i + \delta \|B\|_1 \\ &\leq \bar{B}^i + \delta d \|B\|_\infty \\ &\leq \bar{B}^i + \epsilon \|B\|_\infty. \end{aligned}$$

Therefore $\|A\|_\infty \leq (1 + \epsilon) \|B\|_\infty$. It follows that replacing vectors in I' by those in I increases the height of the machines by only a $(1 + \epsilon)$ factor. $\square$

Large versus small vectors. Assume from here on that we have transformed our instance as described in the above lemma. The second step in the algorithm is to partition the vectors into two sets L and S corresponding to *large* and *small*. L consists of all vectors whose ℓ_∞ norm is greater than δ , and S is the rest of the vectors. The algorithm A_ϵ will have two stages; the first stage packs all the large jobs, and the second stage packs the small jobs. Unlike the case of $d = 1$, the interaction between the two stages has to be taken into account for $d \geq 2$. We show that the interaction can be captured in a compact way as follows. Let $(a_1, a_2, \dots, a_d)$ be a d -tuple of integers such that $0 \leq a_i \leq \lceil 1/\epsilon \rceil$. We will call each such distinct tuple a *capacity configuration*. There are at most $t = (1 + \lceil 1/\epsilon \rceil)^d$ such configurations. Assume that the t capacity configurations (tuples) are ordered in some way and let a_i^k be the value of coordinate i in tuple k . A capacity configuration approximately describes how a

bin is filled. However, we have m bins. A t -tuple $(m_1, \dots, m_t)$, where $0 \leq m_i \leq m$ and $\sum_i m_i = m$, is called a *bin configuration* that describes the number of bins of each capacity configuration. The number of possible bin configurations is clearly $O(m^t)$. Since there are only a polynomial number of such configurations for fixed d and ϵ we can “guess” the configuration used by a feasible packing. A packing of vectors in a bin is said to *respect* a capacity configuration $(a_1, \dots, a_d)$ if the height of the packing in each dimension i is less than ϵa_i . Given a capacity configuration we can define the corresponding *empty capacity configuration* as the tuple obtained by subtracting each entry from $(\lceil 1/\epsilon \rceil + 1)$. For a bin configuration M we denote by $\bar{M}$ the corresponding bin configuration as the one obtained by taking the empty capacity configurations for each of the bins in M .

Overview of the algorithm. The algorithm performs the following steps for each bin configuration M :

- (a) decide if vectors in L can be packed respecting M ,
- (b) decide if vectors in S can be packed respecting $\bar{M}$.

If both steps above succeed for some M , we have a packing of height at most $(1 + \epsilon)$. Otherwise we will prove that the assumption that the optimal packing has a height of 1 is false.

Packing the large vectors. The first stage consists of packing the vectors in L . Observe that the smallest nonzero coordinate of the vectors in L is at least δ^2 . We partition the interval $[\delta^2, 1]$ into $q = \lceil \frac{2}{\epsilon} \ln \delta^{-1} \rceil$ intervals of the form $(x_0, (1 + \epsilon)x_0]$, $(x_1, (1 + \epsilon)x_1]$, $\dots$, $(x_{q-1}, 1]$, where $x_0 = \delta^2$ and $x_{i+1} = (1 + \epsilon)x_i$. We discretize every nonzero coordinate of the vectors in L by rounding the coordinate down to the left end point of the interval in which it falls. Let L' be the resulting set of vectors.

LEMMA 2.2. *Let I' be an instance obtained from the original instance I by rounding vectors in L as described above. Then, replacing each vector in L' by the corresponding vector in L in any solution for I' results in a solution for I of height at most $(1 + \epsilon)$ times that of I' .*

Proof. Each coordinate of a vector in L' is at least $(1 + \epsilon)^{-1}$ times the coordinate of the corresponding vector in L . The lemma follows trivially. $\square$

Vectors in L' can be classified into one of $s = (1 + \lceil \frac{2}{\epsilon} \ln \delta^{-1} \rceil)^d$ distinct classes. Any packing of the vectors into one bin can be described as a tuple $(k_1, k_2, \dots, k_s)$, where k_i indicates the number of vectors of the i th class. Note that at most d/δ vectors from L' can be packed in any bin. Therefore $\sum k_i \leq d/\delta$. Thus there are at most $(d/\delta)^s$ configurations. A configuration is *feasible* for a capacity configuration if the vectors described by the configuration can be packed without violating the height constraints described by the capacity configuration. Let C_k denote the set of all configurations of the jobs in L' that are feasible for the k th capacity configuration. From our discussion $|C_k| \leq (d/\delta)^s$.

LEMMA 2.3. *Let $M = (m_1, m_2, \dots, m_t)$ be a bin configuration. There exists an algorithm with running time $O((d/\delta)^s m n^s)$ to decide if there is a packing of the jobs in L' that respects M .*

Proof. We use a simple dynamic programming-based algorithm. Observe that the number of vector classes in L' is at most s . Thus any subset of vectors from L' can be specified by a tuple of size s , and there are $O(n^s)$ distinct tuples. The algorithm orders bins in some arbitrary way and assigns to each bin a capacity configuration from M . For $1 \leq i \leq m$, the algorithm computes all possible subsets of vectors from L' (tuples) that can be packed validly in the first i bins. For each i this information can be maintained in $O(n^s)$ space. Given the tuples for bin i , the tuples for bin $(i + 1)$ can be computed in $O(d/\delta)^s$ time per tuple since that is an upper bound on the

number of feasible configurations for any capacity configuration. Thus for each bin i , in $O((d/\delta)^s n^s)$ time, we can compute the tuples that can be packed into the first i bins given the information for bin $(i-1)$. The number of bins is m so we get the required time bound. $\square$

Packing the small vectors. We now describe the second stage, that of packing the vectors in S . For the second stage we write an integer programming formulation and round the resulting LP relaxation to find an approximate feasible solution. Without loss of generality assume that the vectors in S are numbered 1 to $|S|$. The IP formulation has 0-1 variables x_{ij} for $1 \leq i \leq |S|$ and $1 \leq j \leq m$. Variable x_{ij} is 1 if p_i is assigned to machine j . Every vector has to be assigned to some machine. This results in the following equation.

$$(2.1) \quad \sum_j x_{ij} = 1, \quad 1 \leq i \leq |S|.$$

Given a bin configuration M , we can define for each machine j and dimension k a height bound b_j^k that an assignment should satisfy. Thus we obtain

$$(2.2) \quad \sum_i p_i^k \cdot x_{ij} \leq b_j^k, \quad 1 \leq j \leq m, \quad 1 \leq k \leq d.$$

In addition we have the integrality constraints, namely, $x_{ij} \in \{0, 1\}$. We obtain a linear program by replacing these constraints by

$$(2.3) \quad x_{ij} \geq 0.$$

PROPOSITION 2.4. *Any basic feasible solution to the LP defined by (2.1), (2.2), and (2.3) has at most $d \cdot m$ vectors that are assigned fractionally to more than one machine.*

Proof. The number of variables in our LP is $n \cdot m$. The number of nontrivial constraints (those that are other than $x_{ij} \geq 0$) is $(n + d \cdot m)$. From standard polyhedral theory [28] any basic (vertex) solution to our LP has $n \cdot m$ tight constraints. Therefore by a simple counting argument, at most $(n + d \cdot m)$ variables can be strictly positive. Since each vector is assigned to at least one machine, the number of vectors that are fractionally assigned to more than one machine is at most $d \cdot m$. $\square$

We can solve the above linear program in polynomial time and obtain a basic feasible solution. Let S' be the set of vectors that are not assigned integrally to any machine. By the above lemma, $|S'| \leq d \cdot m$. We partition the set S' into m sets of at most d vectors each in an arbitrary manner and assign the i th set to the i th machine. Since $\|p_j\|_\infty \leq \delta = \epsilon/d$ for every $j \in S'$, the above step does not violate the height by more than ϵ in any dimension.

Putting together all the ingredients we obtain our main theorem below.

THEOREM 2.5. *Given any fixed $\epsilon > 0$, there is a $(1 + \epsilon)$ -approximation algorithm for the VS problem that runs in $(nd/\epsilon)^{O(s)}$ time, where $s = O((\frac{\ln(d/\epsilon)}{\epsilon})^d)$.*

Proof. Given a correct guess for the optimal schedule height it is clear from the description that we obtain a $(1 + O(\epsilon))$ -approximation. Following the overview of the algorithm we find a packing of vectors in L and S for each choice of bin configuration M . The running time is dominated by the time to pack L . Since there are at most $m^t = O(n^{O(\epsilon^{-d})})$ bin configurations, the running time follows from Lemma 2.3. We can guess the optimal value to within a $(1 + \epsilon)$ precision using the estimate provided by a simple $(d + 1)$ -approximation algorithm described in section 2.2. $\square$

2.2. The general case. We now consider the case when d is arbitrary and present two approximation algorithms for this case. The first algorithm is deterministic and has an approximation ratio that is a function of only d ($O(\ln^2 d)$), while the second algorithm is randomized and achieves an approximation ratio that is a function of both d and m ($O(\ln dm / \ln \ln dm)$). Given a set of positive vectors A we denote by $\mathcal{V}(A)$ the *volume* of A which is the sum of all coordinates of all vectors in A , in other words, the ℓ_1 norm of $\sum_{j \in A} p_j$. We once again assume that the optimal schedule height is 1.

2.2.1. An $O(\ln^2 d)$ -approximation. We start by analyzing a simple algorithm which will serve as a base case for our $O(\ln^2 d)$ -approximation algorithm. Recall that J is the set of vectors in the input instance. The infinity norm of each of the job vectors is clearly a lower bound on the optimal value. Hence

$$(2.4) \quad \max_{j \in J} \|p_j\|_\infty \leq 1.$$

The second lower bound is obtained by using the average volume per dimension:

$$(2.5) \quad \frac{\mathcal{V}(J)}{m \cdot d} \leq 1.$$

We can strengthen the above bound by splitting the sum dimension-wise:

$$(2.6) \quad \max_{i=1}^d \frac{\bar{J}^i}{m} \leq 1.$$

A naive algorithm for our problem is to ignore the multidimensional aspect of the jobs and treat them as one-dimensional vectors of size equal to the sum of their components. The dimensionality of the bins is also ignored. Then one can apply the standard list scheduling algorithm of Graham [16] for multiprocessor scheduling to obtain the following theorem that uses the simple lower bounds developed above.

LEMMA 2.6. *Applying list scheduling on the volumes of the vectors results in a schedule of height at most $\frac{\mathcal{V}(J)}{m} + \max_{j \in J} \|p_j\|_\infty$. This yields a $(d+1)$ -approximation.*

Proof. The upper bound follows from standard analysis of list scheduling. The approximation guarantee follows from the lower bounds in (2.4) and (2.5). $\square$

The $O(\ln^2 d)$ -approximation algorithm. Before we state the algorithm formally we need a couple of definitions. The following problem is a special case of a general PIP.

DEFINITION 2.7. *Given a set J of n vectors in $[0, 1]^d$, the largest volume packing problem is the problem of finding a subset S such that $\|\bar{S}\|_\infty \leq 1$ and $\mathcal{V}(S)$ is maximized. Let $\mathcal{V}_{\max}$ denote the value of the optimal solution.*

DEFINITION 2.8. *An (α, β) -approximation to the largest volume packing problem is a subset S that satisfies the conditions $\|\bar{S}\|_\infty \leq \alpha$ and $\mathcal{V}(S) \geq \beta \mathcal{V}_{\max}$.*

We will typically use the above definition with $\alpha \geq 1$ and $\beta \leq 1$.

Algorithm Volume Pack:

1. **repeat** for t stages
 - (a) **for** $k = 1$ to m **do**
 - i. Find an (α, β) -approximation to the largest volume packing problem with the current set of job vectors.
 - ii. Allocate jobs in packing to machine k and remove them.
2. Find a separate schedule for the remaining jobs using naive volume based list scheduling.
3. Combine the two schedules machine by machine in the obvious way.

We now prove several simple lemmas to analyze the performance of the above algorithm.

LEMMA 2.9. *Let $J(i)$ be the set of jobs remaining at the beginning of the i th stage with $J(1) = J$. Let $J_k(i)$ be the set of jobs remaining after machine k has been packed in stage i . Then*

$$\mathcal{V}(J_k(i)) \leq \mathcal{V}(J(i)) \cdot (1 - \beta/m)^k.$$

Proof. We prove the lemma by induction on k . The claim is trivially true for $k = 0$. Suppose the claim is true up to machine k . We show that the claim is true for machine $(k + 1)$. Since all jobs in J can be scheduled on m machines with height 1, it follows that all jobs in $J_k(i)$ can be likewise scheduled. By a simple averaging argument we can infer that there exists a set of jobs in $J_k(i)$ with volume at least $\mathcal{V}(J_k(i))/m$ that can be packed in a machine with height at most 1. Since we obtain a β -approximation to largest volume packing, we pack jobs with a volume of at least $\beta \cdot \mathcal{V}(J_k(i))/m$. Therefore $\mathcal{V}(J_{k+1}(i)) \leq \mathcal{V}(J_k(i)) \cdot (1 - \beta/m)$. By our induction hypothesis $\mathcal{V}(J_k(i)) \leq \mathcal{V}(J(i)) \cdot (1 - \beta/m)^k$. The lemma follows. $\square$

COROLLARY 2.10. $\mathcal{V}(J(i)) \leq \mathcal{V}(J)/e^{(i-1)\beta}$.

Proof. From Lemma 2.9, $\mathcal{V}(J(i)) \leq \mathcal{V}(J(i-1)) \cdot (1 - \beta/m)^m$. Since $(1 - \beta/m)^m \leq e^{-\beta}$, we get the required bound. $\square$

LEMMA 2.11. *Algorithm Volume Pack yields a schedule of height at most $(t \cdot \alpha + \frac{d}{e^{t\beta}} + 1)$.*

Proof. Consider the machine that achieves the maximum height in the schedule produced by Volume Pack. Let J_1 and J_2 be the set of jobs allocated to that machine in the packing stage and the list scheduling stage, respectively. From the packing property it is easy to see that the height of machine due to jobs in J_1 is at most $t\alpha$. Let J' be the set of jobs remaining after the t stages of packing. These are scheduled using list scheduling. From Corollary 2.10 we have that

$$\mathcal{V}(J') \leq \mathcal{V}(J)/e^{t\beta}.$$

From Lemma 2.6, the height increase of the machine due to jobs in J_2 is at most

$$\mathcal{V}(J')/m + \max_j \|p_j\|_\infty \leq \frac{d}{e^{t\beta}} \cdot \mathcal{V}(J)/(dm) + 1 \leq \frac{d}{e^{t\beta}} + 1.$$

In the above inequality we are using the fact that the two lower bounds are less than 1, the optimal value. Combining the two equations gives us the desired bound. $\square$

The parameter t in the algorithm can be chosen as a function of α and β to obtain the best ratio. Note that the largest volume packing problem is a special case of a PIP, where c_i is simply the volume of vector i . PIPs have an $(O(\ln d), 1/2)$ -approximation via randomized rounding [27, 29] that can be derandomized by techniques from [26]. When d is fixed there is a $(1, 1 - \epsilon)$ approximation [7] that runs in time polynomial in $n^{d/\epsilon}$. These observations imply the following.

THEOREM 2.12. *There is an $O(\ln^2 d)$ -approximation algorithm for the VS problem.*

THEOREM 2.13. *There is an $O(\ln d)$ -approximation algorithm for the VS problem that runs in time polynomial in n^d .*

2.2.2. An $O(\ln dm / \ln \ln dm)$ -approximation. The approximation result of Theorem 2.12 is good when d is small compared to m . However, when d is large we can obtain an $O(\ln dm / \ln \ln dm)$ -approximation by a simple randomized algorithm

that assigns each vector independently to a machine chosen uniformly at random from the set of m machines. Theorem 2.15 bounds the performance of this algorithm, referred to as *Random*. We need a version of the standard Chernoff bound on sums of independent random variables.

PROPOSITION 2.14. *Let $X_1, \dots, X_n$ be independent binary random variables and let $X = \sum_{i=1}^n t_i X_i$. Let $T = \max_i t_i$ and $\mu = \mathbf{E}[X]$. Then, for any sufficiently large h , $\Pr[X > (1 + c \frac{\ln h}{\ln \ln h})(\mu + T)] \leq h^{-c/2}$.*

Proof. Let Y_i be a random variable such that $Y_i = \frac{t_i}{T} X_i$. Note that Y_i takes on values in $[0, 1]$. Let $Y = \sum_i Y_i$. It follows that $Y = X/T$ and that $\mathbf{E}[Y] = \mu/T$. Therefore, for any $\delta > 0$, $\Pr[X > (1 + \delta)(\mu + T)] = \Pr[Y > (1 + \delta)(\mu/T + 1)]$. Applying the standard Chernoff–Hoeffding bounds [25] on sums of independent random variables that assume values in $[0, 1]$ to Y , we get the desired result. $\square$

THEOREM 2.15. *Random gives an $O(\ln dm / \ln \ln dm)$ -approximation with high probability.*

Proof. Consider the first machine. Let X_j be the indicator random variable that is 1 if vector j is assigned to the first machine. The X_j 's are independent. By uniformity $\Pr[X_j = 1] = 1/m$. Let $P = \sum_j p_j X_j$. Note that P is a vector since each p_j is a vector: let P^i denote the i th coordinate of P . By linearity of expectations, $\mathbf{E}[P^i] = \sum_j p_j^i / m \leq 1$ (using (2.5)). Also observe that $\max_j p_j^i \leq 1$ (using (2.4)). Now we estimate the probability that P^i deviates significantly from its expected value. From Proposition 2.14, $\Pr[P^i > (\mathbf{E}[P^i] + \max_j p_j^i)(1 + c \ln dm / \ln \ln dm)] \leq (dm)^{-c/2}$. Thus with high probability P^i is $O(\ln dm / \ln \ln dm)$. In general, if A_k^i is the event that the i th dimension of machine k is greater than $2(1 + c) \ln dm / \ln \ln dm$, then from above we know that $\Pr[A_k^i] \leq (dm)^{-c/2}$. Thus $\Pr[A = \cup_{i=1}^d \cup_{k=1}^m A_k^i] \leq dm(dm)^{-c/2}$. By choosing c sufficiently large we can ensure that $\Pr[A]$ is less than an inverse polynomial factor. But the complement of A is the event that the schedule length is $O(\ln dm / \ln \ln dm)$. Thus with high probability we get an $O(\ln dm / \ln \ln dm)$ -approximation. $\square$

3. Algorithms for vector bin packing. We now examine the problem of packing a given set of vectors into the smallest possible number of bins. Our main result here is as follows.

THEOREM 3.1. *For any fixed $\epsilon > 0$, we can obtain in polynomial time a $(1 + \epsilon \cdot d + O(\ln(1/\epsilon)))$ -approximate solution for vector bin packing.*

This improves upon the long-standing $(d + \epsilon)$ -approximation algorithm of [6]. Our approach is based on solving a linear programming relaxation for this problem. As in section 2.1, we use a variable x_{ij} to indicate if vector p_i is assigned to bin j . We guess the least number of bins m (easily located via binary search) for which the following LP relaxation is feasible; clearly $m \leq \text{OPT}$.

$$\begin{aligned} \sum_j x_{ij} &= 1, & 1 \leq i \leq n, \\ \sum_i p_i^k \cdot x_{ij} &\leq 1, & 1 \leq j \leq m, \ 1 \leq k \leq d, \\ x_{ij} &\geq 0, & 1 \leq i \leq n, \ 1 \leq j \leq m. \end{aligned}$$

Once again, we use the fact that a basic feasible solution would make fractional bin assignments for at most $d \cdot m$ vectors. Thus at this point, all but a set S of at most $d \cdot m$ vectors have integral assignments in m bins. To find a bin assignment for S , we repeatedly find a set $S' \subseteq S$ of up to $k = \lceil 1/\epsilon \rceil$ vectors that can all be packed together

and assign them to a new bin. This step is performed greedily; i.e., we seek to find a largest possible such set in each iteration. We can perform this step by trying out all possible sets of vectors of cardinality less than $(k + 1)$. We now claim that this procedure must terminate in $\epsilon \cdot d \cdot m + O(\ln \epsilon^{-1}) \cdot \text{OPT}$ steps. To see this, consider the first time that we pack less than k vectors in a bin. The number of bins used thus far is bounded by $(d \cdot m)/k$. Moreover, the total number of vectors that remain at this point is at most $(k - 1)\text{OPT}$; let S' denote this remaining set of vectors. Since the optimal algorithm cannot pack more than $(k - 1)$ vectors of S' in one bin, our greedy bin assignment procedure is identical to a greedy set cover algorithm, where each set has size at most $(k - 1)$. Following the analysis of the greedy algorithm in [17], the total number of bins used in packing vectors in S' is bounded by $H_{k-1} \cdot \text{OPT}$ (H_i is the i th harmonic number). Putting things together, we obtain that the number of bins used by our algorithm, A , is bounded as follows:

$$A \leq m + (d \cdot m)/k + H_{k-1} \cdot \text{OPT} \leq (1 + \epsilon \cdot d + O(\ln \epsilon^{-1})) \cdot \text{OPT}.$$

This completes the proof of Theorem 3.1. Substituting $\epsilon = 1/d$, we obtain the following simple corollary.

COROLLARY 3.2. *For fixed d , VBP can be approximated to within $O(\ln d)$ in polynomial time.*

4. Inapproximability results. In this section we show hardness of approximation results for the three problems we consider, vector scheduling, vector bin packing, and packing integer programs. We start with PIPs.

4.1. Packing integer programs. Randomized rounding techniques of Raghavan and Thompson [27] yield integral solutions of value $t_1 = \Omega(\text{OPT}/d^{1/B})$ if $A \in [0, 1]^{d \times n}$, and $t_2 = \Omega(\text{OPT}/d^{1/(B+1)})$ if $A \in \{0, 1\}^{d \times n}$. Srinivasan [29] improved these results to obtain solutions of value $\Omega(t_1^{B/(B-1)})$ and $\Omega(t_2^{(B+1)/B})$, respectively. We show that PIPs are hard to approximate to within a factor of $\Omega(d^{\frac{1}{B+1}-\epsilon})$ for every fixed integer B . We start with the case $A \in \{0, 1\}^{d \times n}$ and then indicate how our result extends to $A \in [0, 1]^{d \times n}$. Our reduction uses the result of Hastad [18] that shows that unless $NP = ZPP$ the maximum independent set problem is hard to approximate within a factor of $n^{1-\epsilon}$ for any fixed $\epsilon > 0$. Since the upper bounds are in terms of d , from here on, we will express the inapproximability factor only as a function of d .

Given a graph $G = (V, E)$ with $|V| = n$ and a positive integer B , we construct an instance of a PIP, I_G , as follows. Let A be a $d \times n$ 0-1 matrix with $d = n^{(B+1)}$ such that each row corresponds to an element from $V^{(B+1)}$. Let $r_i = (v_{i_1}, \dots, v_{i_{(B+1)}})$ denote the tuple associated with the i th row of A . We set $a_{ij} = 1$ if and only if the following conditions hold, otherwise we set it to 0: (a) the vertex v_j occurs in r_i , and (b) the vertices in r_i induce a *clique* in G .

We set $c = \{1\}^n$ and $b = \{B\}^d$. For any fixed integer B , the reduction can be done in polynomial time. Note that a feasible solution to I_G can be described as a set of indices $S \subseteq \{1, \dots, n\}$.

LEMMA 4.1. *Let $X \subseteq V$ be an independent set of G . Then $S = \{i \mid v_i \in X\}$ is a feasible solution to I_G of value $|S| = |X|$. Furthermore, S can be packed with a height bound of 1.*

Proof. Suppose that in some dimension the height induced by S is greater than 1. Let r be the tuple associated with this dimension. Then there exist $i, j \in S$ such that $v_i, v_j \in r$ and $(v_i, v_j) \in E$. This contradicts the assumption that X is an independent set. $\square$

LEMMA 4.2. *Let S be any feasible solution to I_G and let G_S be the subgraph of G induced by the set of vertices v_i such that $i \in S$. Then $\omega(G_S) \leq B$, where $\omega(G_S)$ is the clique number of G_S .*

Proof. Suppose there is a clique of size $(B + 1)$ in G_S ; without loss of generality assume that $v_1, \dots, v_{(B+1)}$ are the vertices of that clique. Consider the tuple $(v_1, v_2, \dots, v_{(B+1)})$ and let i be the row of A corresponding to the above tuple. Then by our construction, $a_{ij} = 1$ for $1 \leq j \leq (B + 1)$. There are $(B + 1)$ vectors in S with a 1 in the same dimension i , violating the i th row constraint. This contradicts the feasibility of S . $\square$

The following is a standard Ramsey type result.

LEMMA 4.3. *Let G be a graph on n vertices with $\omega(G) \leq k$. Then $\alpha(G) \geq n^{1/k}$.*

Lemmas 4.2 and 4.3 give us the following corollary.

COROLLARY 4.4. *Let S be any valid solution to I_G of value $t = |S|$. Then $\alpha(G) \geq t^{1/B}$.*

THEOREM 4.5. *Unless $NP = ZPP$, for every fixed integer B and fixed $\epsilon_0 > 0$, PIPs with bound $b = \{B\}^d$ and $A \in \{0, 1\}^{d \times n}$ are hard to approximate to within a factor of $d^{\frac{1}{B+1} - \epsilon_0}$. PIPs with $A \in [0, 1]^{d \times n}$ and B rational are hard to approximate to within a factor of $d^{\frac{1}{\lfloor B \rfloor + 1} - \epsilon_0}$.*

Proof. We first look at the case of PIPs with $A \in \{0, 1\}^{d \times n}$. Notice that our reduction produces only such instances. Suppose there is a polynomial time approximation algorithm $\mathcal{A}$ for PIPs with bound B that has an approximation ratio $d^{\frac{1}{B+1} - \epsilon_0}$ for some fixed $\epsilon_0 > 0$. This can be reinterpreted as a $d^{\frac{1}{B+1} - \epsilon}$ -approximation, where $\epsilon = \epsilon_0(B + 1)$ is another constant. We will obtain an approximation algorithm $\mathcal{G}$ for the maximum independent set problem with a ratio $n^{1-\delta}$ for $\delta = \epsilon/B$. The hardness of maximum independent [18] will then imply the desired result. Given a graph G , the algorithm $\mathcal{G}$ constructs an instance I_G of a PIP as described above and gives it as input to $\mathcal{A}$. $\mathcal{G}$ returns $\max(1, t^{1/B})$ as the independent set size of G , where t is the value returned by $\mathcal{A}$ on I_G . Note that by Corollary 4.4, $\alpha(G) \geq t^{1/B}$, which proves the correctness of the algorithm. Now we prove the approximation guarantee. We are interested only in the case when $\alpha(G) \geq n^{1-\delta}$, for otherwise a trivial independent set of size 1 gives the required approximation ratio. From Lemma 4.1 it follows that the optimal value for I_G is at least $\alpha(G)$. Since $\mathcal{A}$ provides a $d^{\frac{1-\epsilon}{B+1}}$ -approximation, $t \geq \alpha(G)/d^{\frac{1-\epsilon}{B+1}}$. In the construction of I_G , $d = n^{(B+1)}$. Therefore $t \geq \alpha(G)/n^{(1-\epsilon)}$. Simple algebra verifies that $t^{1/B} \geq \alpha(G)/n^{1-\delta}$ when $\alpha(G) \geq n^{1-\delta}$.

Now we consider the case of PIPs with $A \in [0, 1]^{d \times n}$. Let B be some real number. For a given B we can create an instance of a PIP as before with $B' = \lfloor B \rfloor$. The only difference is that we set $b = B^d$. Since all entries of A are integral, effectively the bound is B' . Therefore it is hard to approximate to within a factor of $d^{(1-\epsilon)/(B'+1)} = d^{(1-\epsilon)/(\lfloor B \rfloor + 1)}$. Since $(\lfloor B + 1/d \rfloor + 1) = B + 1$, $d^{(1-\epsilon)/(\lfloor B \rfloor + 1)} = \Theta(d^{(1-\epsilon)/B})$. $\square$

Discussion. An interesting aspect of our reduction above is that the hardness results hold even when the optimal algorithm is restricted to a height bound of 1 while allowing a height bound of B for the approximation algorithm. Let an (α, β) -bicriteria approximation be one that satisfies the relaxed constraint matrix $Ax \leq \alpha b$ and gets a solution of value at least OPT/β ; here OPT satisfies $Ax \leq b$. Then we have the following corollary.

COROLLARY 4.6. *Unless $NP = ZPP$, for every fixed integer B and fixed $\epsilon > 0$, it is hard to obtain a $(B, d^{\frac{1}{B+1} - \epsilon})$ bicriteria approximation for PIPs.*

For a given B , we use $d = n^{B+1}$, and a hardness of $d^{\frac{1}{B+1} - \epsilon}$ is essentially the

hardness of $n^{1-\epsilon}$ for the independent set problem. This raises two related questions. First, should d be larger than n to obtain the inapproximability results? Second, should the approximability (and inapproximability) results be parameterized in terms of n instead of d ? These questions are important to understand the complexity of PIPs as d varies from $O(1)$ to $\text{poly}(n)$. We observe that the hardness result holds as long as d is $\Omega(n^\epsilon)$ for some fixed $\epsilon > 0$. To see this, observe that in our reduction, we can always add $\text{poly}(n)$ dummy columns (vectors) that are either useless (their c_j value is 0) or cannot be packed (add a dummy dimension where only B of the dummy vectors can be packed). Thus we can ensure that $n \geq \text{poly}(d)$ without changing the essence of the reduction. We have a PTAS when $d = O(1)$ and a hardness result of $d^{1/(B+1)}$ when $d = \text{poly}(n)$. An interesting question is to resolve the complexity of the problem when $d = \text{polylog}(n)$.

As remarked earlier, Srinivasan [29] improves the results obtained using randomized rounding to obtain solutions of value $\Omega(t_2^{(B+1)/B})$, where $t_2 = \Omega(y^*/d^{1/(B+1)})$ for $A \in \{0, 1\}^{d \times n}$. In the above y^* is the optimal fractional solution to the PIP. It might appear that this contradicts our hardness result but observe that for the instances we create in our reduction $y^*/d^{1/(B+1)} \leq 1$. For such instances Srinivasan's bounds do not yield an improvement over randomized rounding.

4.2. Vector scheduling. We now extend the ideas used in the hardness result for PIPs to show the following hardness result for the VS problem.

THEOREM 4.7. *Unless $NP = ZPP$, for every constant $\gamma > 1$, there is no polynomial time algorithm that approximates the schedule height in the VS problem to within a factor of γ .*

Our result here uses the hardness of graph coloring; Feige and Kilian [5] building on the work of Hastad [18] show that graph coloring is $n^{1-\epsilon}$ -hard unless $NP = ZPP$. Our reduction is motivated by the fact that graph coloring corresponds to covering a graph by independent sets. We start with the following simple lemma that is easily derived from Lemma 4.3.

LEMMA 4.8. *Let G be a graph on n vertices with $\omega(G) \leq k$. Then $\chi(G) \leq O(n^{1-1/k} \ln n)$.*

Let $B = \lceil \gamma \rceil$; we will show that it is hard to obtain a B -approximation using a reduction from chromatic number. Given graph G we construct an instance I of the VS problem as follows. We construct n vectors of n^{B+1} dimensions as in the proof of Theorem 4.5. We set m , the number of machines, to be $n^{\frac{1}{2B}}$.

LEMMA 4.9. *If $\chi(G) \leq m$, then the optimal schedule height for I is 1.*

Proof. Let $V_1, \dots, V_{\chi(G)}$ be the color classes. Each color class is an independent set, and by Lemma 4.1 the corresponding vectors can be packed on one machine with height at most 1. Since $\chi(G) \leq m$, the vectors corresponding to each color class can be packed in a separate machine. $\square$

LEMMA 4.10. *If the schedule height for I is bounded by B , then $\chi(G) \leq \beta n^{1-1/2B} \ln n$ for some fixed constant β .*

Proof. Let $V_1, V_2, \dots, V_m$ be the partition of vertices of G induced by the assignment of the vectors to the machines. Let G_i be the subgraph of G induced by the vertex set V_i . From Lemma 4.2 we have $\omega(G_i) \leq B$. Using Lemma 4.8 we obtain that $\chi(G_i) \leq \beta n^{1-1/B} \ln n$ for $1 \leq i \leq m$. Therefore it follows that $\chi(G) \leq \sum_i \chi(G_i) \leq m \cdot \beta n^{1-1/B} \ln n \leq \beta n^{1-1/2B} \ln n$. $\square$

Proof of Theorem 4.7. Feige and Kilian [5] showed that unless $ZPP = NP$ for every $\epsilon > 0$ there is no polynomial time algorithm to approximate the chromatic number to within a factor of $n^{1-\epsilon}$. Suppose there is a B -approximation for the

VS problem. Lemmas 4.9 and 4.10 establish that if $\chi(G) \leq n^{1/2B}$, then we can infer by running the B -approximation algorithm for the VS problem that $\chi(G) \leq \beta n^{1-1/2B} \ln n$. This implies a $\beta n^{1-1/2B} \ln n$ -approximation to the chromatic number. From the result of [5] it follows that this is not possible unless $NP = ZPP$. $\square$

4.3. Vector bin packing. As mentioned before, VBP is APX-hard even for $d = 2$. We show a simple $d^{\frac{1}{2}-\epsilon}$ hardness for VBP when d is arbitrary. The reduction is similar to that in the previous subsection and uses hardness of graph coloring. Given a graph G with n vertices and m edges we create an instance I_G of VBP with n vectors, each of m dimensions. For a vector v_i corresponding to a vertex i of G , the j th coordinate is 1 if i is incident on the j th edge, otherwise it is 0. If the vectors are required to be packed into bins of height 1, it is easily seen that the number of bins required corresponds exactly to a coloring of G . Thus the hardness of chromatic number applies directly to VBP.

THEOREM 4.11. *Unless $NP = ZPP$, VBP is hard to approximate to within a $d^{\frac{1}{2}-\epsilon}$ factor for every fixed $\epsilon > 0$.*

5. Conclusions. We studied multidimensional generalizations of multiprocessor scheduling, bin packing, and the knapsack problem and obtained a variety of new algorithmic as well as inapproximability results for them. While our work gives new insights into the approximability of these problems, several questions remain open. In particular, large gaps remain between the upper and lower bounds for both VS and VBP when the number of dimensions is arbitrary. For packing integer programs, our hardness result is essentially tight, but it applies only to fixed values of B . The quality of approximation improves dramatically when B is allowed to grow as a logarithmic function of d ; in particular a constant factor approximation is achievable when $B = \Omega(\log d / \log \log d)$. It will be interesting to see if our techniques can be extended to obtain a hardness result when B is allowed to be a function of d .

Acknowledgments. We are grateful to Aravind Srinivasan for guiding us through the literature on packing integer programs and his encouragement at the early stages of this research. We thank Cliff Stein for his comments on this work in its form as a chapter in the dissertation of the first author. We also thank the anonymous referees for many useful comments on an earlier version of the paper.

REFERENCES

- [1] N. ALON, Y. AZAR, J. CSIRIK, L. EPSTEIN, S. V. SEVASTIANOV, A. P. A. VESTJENS, AND G. J. WOEGINGER, *On-line and off-line approximation algorithms for vector covering problems*, Algorithmica, 21 (1998), pp. 104–118.
- [2] S. F. ASSMAN, *Problems in Discrete Applied Mathematics*, Ph.D. thesis, Mathematics Department, Massachusetts Institute of Technology, Cambridge, MA, 1983.
- [3] C. CHEKURI AND S. KHANNA, *On multi-dimensional packing problems*, in Proceedings of the 10th Annual ACM-SIAM Symposium on Discrete Algorithms, ACM, New York, SIAM, Philadelphia, 1999, pp. 185–194.
- [4] D. J. DEWITT AND J. GRAY, *Parallel database systems: The future of high performance database systems*, Comm. ACM, 35 (1992), pp. 85–98.
- [5] U. FEIGE AND J. KILIAN, *Zero knowledge and the chromatic number*, J. Comput. System Sci., 57 (1998), pp. 187–199.
- [6] W. FERNANDEZ DE LA VEGA AND G. S. LUEKER, *Bin packing can be solved within $1 + \epsilon$ in linear time*, Combinatorica, 1 (1981), pp. 349–355.
- [7] A. M. FRIEZE AND M. R. B. CLARKE, *Approximation algorithms for the m -dimensional 0-1 knapsack problem: Worst-case and probabilistic analyses*, European J. Oper. Res., 15 (1984), pp. 100–109.

- [8] M. R. GAREY AND R. L. GRAHAM, *Bounds for multiprocessor scheduling with resource constraints*, SIAM J. Comput., 4 (1975), pp. 187–200.
- [9] M. R. GAREY, R. L. GRAHAM, D. S. JOHNSON, AND A. C. YAO, *Resource constrained scheduling as generalized bin packing*, J. Combin. Theory Ser. A, 21 (1976), pp. 257–298.
- [10] M. R. GAREY AND D. S. JOHNSON, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, New York, 1979.
- [11] M. N. GAROFALAKIS AND Y. E. IOANNIDIS, *Scheduling issues in multimedia query optimization*, ACM Computing Surveys, 27 (1995), pp. 590–592.
- [12] M. N. GAROFALAKIS AND Y. E. IOANNIDIS, *Multi-dimensional resource scheduling for parallel queries*, in Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data, 1996, pp. 365–376.
- [13] M. N. GAROFALAKIS AND Y. E. IOANNIDIS, *Parallel query scheduling and optimization with time-and space-shared resources*, in Proceedings of the 23rd International Conference on Very Large Data Bases, Morgan Kaufmann, San Francisco, 1997, pp. 296–305.
- [14] M. N. GAROFALAKIS, B. ÖZDEN, AND A. SILBERSCHATZ, *Resource scheduling in enhanced paper-view continuous media databases*, in Proceedings of the 23rd International Conference on Very Large Data Bases, Morgan Kaufmann, San Francisco, 1997, pp. 516–525.
- [15] R. L. GRAHAM, *Bounds for certain multiprocessor anomalies*, Bell System Tech. J., 45 (1966), pp. 1563–1581.
- [16] R. L. GRAHAM, *Bounds on multiprocessor timing anomalies*, SIAM J. Appl. Math., 17 (1969), pp. 416–429.
- [17] M. M. HALLDÓRSSON, *Approximating k -set cover and complementary graph coloring*, in Proceedings of the Fifth IPCO Conference on Integer Programming and Combinatorial Optimization, Lecture Notes in Comput. Sci. 1084, Springer-Verlag, Berlin, 1996, pp. 118–131.
- [18] J. HÅSTAD, *Clique is hard to approximate to within $n^{1-\epsilon}$* , in Proceedings of the 37th Annual Symposium on Foundations of Computer Science, IEEE Comput. Soc. Press, Los Alamitos, CA, 1996, pp. 627–636.
- [19] D. S. HOCHBAUM AND D. B. SHMOYS, *Using dual approximation algorithms for scheduling problems: Theoretical and practical results*, J. ACM, 34 (1987), pp. 144–162.
- [20] O. H. IBARRA AND C. E. KIM, *Fast approximation algorithms for the knapsack and sum of subset problems*, J. ACM, 22 (1975), pp. 463–468.
- [21] N. KARMAKAR AND R. KARP, *An efficient approximation scheme for the one-dimensional bin-packing problem*, in Proceedings of the 23rd Annual Symposium on Foundations of Computer Science, IEEE, New York, 1982, pp. 312–320.
- [22] R. M. KARP, M. LUBY, AND A. MARCHETTI-SPACCAMELA, *A probabilistic analysis of multi-dimensional bin packing problems*, in Proceedings of the Annual ACM Symposium on the Theory of Computing, 1984, pp. 289–298.
- [23] E. L. LAWLER, J. K. LENSTRA, A. H. G. RINNOOY KAN, AND D. B. SHMOYS, *Sequencing and scheduling: Algorithms and complexity*, in Handbooks in Operations Research and Management Science, Vol. 4, Elsevier Science, Amsterdam, 1993, pp. 445–522.
- [24] L. LOVÁSZ, *On the ratio of the optimal integral and fractional covers*, Discrete Math., 13 (1975), pp. 383–390.
- [25] R. MOTWANI AND P. RAGHAVAN, *Randomized Algorithms*, Cambridge University Press, Cambridge, UK, 1995.
- [26] P. RAGHAVAN, *Probabilistic construction of deterministic algorithms: Approximating packing integer programs*, J. Comput. System Sci., 37 (1988), pp. 130–143.
- [27] P. RAGHAVAN AND C. D. THOMPSON, *Randomized rounding: A technique for provably good algorithms and algorithmic proofs*, Combinatorica, 7 (1987), pp. 365–374.
- [28] A. SCHRIJVER, *Theory of Linear and Integer Programming*, Wiley-Interscience Series in Discrete Mathematics, Wiley, Chichester, UK, 1986.
- [29] A. SRINIVASAN, *Improved approximations of packing and covering problems*, in Proceedings of the 27th ACM Symposium on the Theory of Computing, 1995, pp. 268–276.
- [30] G. WOEINGING, *A polynomial time approximation scheme for maximizing the minimum completion time*, Oper. Res. Lett., 20 (1997), pp. 149–154.
- [31] G. WOEINGING, *There is no asymptotic PTAS for two-dimensional vector packing*, Inform. Process. Lett., 64 (1997), pp. 293–97.